

Working with Claude Code

Plan mode as a permission boundary, the one-sentence test, and why a failing test beats a description of the bug.

Plan mode is a **permission mode**. Edits are blocked at the tool layer, not discouraged by a prompt.

Explore, plan, code, commit

01

Explore

Read the relevant code.
Understand before
proposing.

02

Plan

Produce an approach
you can review and
revise.

03

Code

Implement against the
approved plan.

04

Commit

Descriptive message,
and a pull request.

ON THE EXAM

Plan mode covers the first two phases. The value is that you get to disagree before anything exists.

Getting in and out

- ◆ **Shift and Tab cycles the permission modes.** Default, then accept edits, then plan.
- ◆ **/plan** enters it for a single prompt, without flipping the whole session.
- ◆ **--permission-mode plan** starts a session in it, and a settings default makes a project plan-first.
- ◆ **Approving a plan exits plan mode** and moves you into whichever mode your approval option describes.
- ◆ **Shift and Tab again leaves without approving.** Entering does not commit you to producing a plan.

ON THE EXAM

Four routes in. The settings default is the one to reach for on a repository where planning should always happen.

If you could describe the diff in **one sentence**, skip the plan.

Plan first, or go directly?

PLAN MODE

The change touches several files

You are unsure of the approach

The code is unfamiliar to you

Production paths or migrations

The instruction is ambiguous

STRAIGHT IN

One file, one obvious edit

The approach is not in question

You wrote it last week

A log line, a typo, a rename

Tight iteration on something small

ON THE EXAM

Uncertainty, breadth and risk push towards planning. Clarity and smallness push against it.

Plan mode, or asking for a plan?

PLAN MODE

A boundary

- Edits blocked at the tool layer
- Cannot start implementing early
- Approval is an explicit gate
- The plan is a reviewable artefact

"MAKE A PLAN FIRST"

A request

- An instruction in the prompt
- Usually obeyed, sometimes not
- No gate; it flows straight into edits
- Fine when nothing is at stake

THE TIEBREAKER

The same difference as everywhere else in this course. One is enforced; the other is asked for.

The plan is an artefact

- ◆ **Plans are written to disk**, so they survive compaction and session restarts.
- ◆ **Revise before approving.** Add steps, remove steps, reorder. Nothing has happened yet.
- ◆ **Iterating on a plan is far cheaper than iterating on a diff.**
- ◆ **For a significant feature the plan becomes the pull request description**, so reviewers see intent next to implementation.

ON THE EXAM

Course-correcting during planning costs a sentence. Course-correcting after implementation costs a rewrite.

Without an external check, the agent's only verifier is its own judgement — and that **degrades as context fills.**

Describe the bug, or write the failing test?

DESCRIBE IT

"The date parser breaks on leap years"

- Claude must infer the exact failure
- Success is judged by reading the code
- Nothing tells you when it is fixed
- Regressions are invisible

A FAILING TEST

A test that fails now and passes after

- The failure is unambiguous and reproducible
- Success is a green run, not an opinion
- The loop terminates on evidence
- The test stays as a regression guard

THE TIEBREAKER

A failing test converts "does this look right?" into "did it pass?". Those are different questions.

The iteration loop that works

- ◆ **Write the test first**, or ask Claude to write one that reproduces the problem.
- ◆ **Confirm it fails** for the right reason, before anything is fixed.
- ◆ **Let Claude implement**, then run the suite.
- ◆ **Feed failures back** rather than describing them. The output is better evidence than your summary.
- ◆ **The loop ends when the suite is green**, which is a fact rather than a judgement.

ON THE EXAM

This is module 1's four-step loop with the verification step made concrete.

The same trick, other domains

- ◆ **Front-end work:** give it a **design mock** and a way to screenshot, so it can compare and iterate.
- ◆ **Data work:** an **assertion on the output** beats a description of what the output should look like.
- ◆ **Infrastructure:** a **plan or a dry run** that can be diffed against the intended state.
- ◆ **The pattern is always the same.** Something outside the model that returns pass or fail.

ON THE EXAM

Anthropic reports the visual compare loop producing a two to three times quality improvement on front-end work.

Getting the request right

- ◆ **Ask Claude to interview you** when the requirement is fuzzy. Questions first, code second.
- ◆ **Reference specific files with @** for one-off context, rather than adding them to a memory file.
- ◆ **Paste a working reference.** Claude does better with a concrete example than an abstract description.
- ◆ **Fix coupled issues in dependency order**, not in the order you noticed them.

ON THE EXAM

An @ reference is for this conversation. CLAUDE.md is for every conversation. Do not confuse the two.

One anti-pattern worth naming

- ◆ **Do not block Edit or Write mid-implementation** with a hook that rejects work in progress.
- ◆ **Blocking at write time breaks multi-step reasoning**, because Claude loses its place in the sequence.
- ◆ **Let it finish, then validate.** A PostToolUse hook, or a pre-commit check.
- ◆ **Prevention belongs at the boundary**, not in the middle of a task.

ON THE EXAM

Module 9 said prohibitions go in PreToolUse. This is the exception's shape: style rules are not prohibitions.

Reading the stem: workflow questions

WHAT THE STEM SAYS	WHAT IT WANTS
"started editing before anyone agreed the approach"	Plan mode. The boundary, not a firmer instruction.
"a one-line change"	Skip the plan. Planning is overhead here.
"nobody can tell whether it is actually fixed"	A failing test, or another external check
"unfamiliar codebase, multi-file refactor"	Plan mode, every time
"it loses track part way through a large change"	Check for a hook blocking writes mid-task
"the requirement is ambiguous"	Have Claude ask questions before implementing

ON THE EXAM

Size and certainty decide the planning questions. Evidence decides the iteration questions.

Currency check

TOPIC	EXAM ANSWER	CURRENT REALITY
Entering plan mode	Shift+Tab	Also /plan, --permission-mode plan, and a settings default.
What runs during planning	Reads only	Reads plus a read-only command set; auto mode can classify others.
Default permission mode	Default mode	Auto mode became the default for new sessions on paid plans in August 2026.

ON THE EXAM

The exam-era framing is still correct: plan mode is read-only and edits are blocked. The routes in have multiplied.

Module 10 in five lines

- ◆ Plan mode is a permission mode. Edits are blocked at the tool layer, not discouraged.
- ◆ If you could describe the diff in one sentence, skip the plan.
- ◆ Uncertainty, breadth and risk favour planning. Clarity and smallness favour going straight in.
- ◆ A failing test turns "does this look right?" into "did it pass?".
- ◆ Without an external oracle, the agent grades its own work with judgement that degrades as context fills.

ON THE EXAM

Five scenarios next. Two are the plan-or-not decision pointing opposite ways.

An engineer asks Claude Code to add rate limiting to an internal API. The codebase is new to them. The change will touch the middleware layer, the configuration schema, three route handlers and the test suite.

Claude begins editing files immediately. Twenty minutes later the engineer realises it has put the rate limiter in the wrong layer, and the work has to be unpicked.

They want to prevent this recurring.

What should they change?

- ☐ A Add a line to CLAUDE.md telling Claude to propose an approach before making changes
- ☐ B Use plan mode for changes of this shape, and approve the plan before implementation begins
- ☐ C Ask Claude to explain its approach in the prompt before it starts
- ☐ D Add a PreToolUse hook blocking Edit until a plan file exists

◆ PAUSE AND DECIDE

What should they change?

- A

A line in CLAUDE.md

Context. It will work most of the time, which is what they already have.
- B

Plan mode

Unfamiliar code, several files, and an approach worth disagreeing with. All three triggers, and edits are blocked until approval.
- C

Ask in the prompt

A request with no gate. Claude explains, then implements immediately anyway.
- D

Hook blocking Edit

Enforcement, aimed badly. It blocks writes mid-task and breaks the multi-step reasoning it was meant to protect.

THE SIGNAL

Unfamiliar code, multiple files, an arguable approach. Three planning triggers in one paragraph.

The same engineer now sets plan mode as the project default in settings.

Over the following week they notice that renaming a variable, adding a log line and correcting a typo in an error message each produce a three-paragraph plan they have to read and approve.

They estimate the overhead at several minutes a day.

What should they do?

- ☐ A Keep the default. Planning is always safer than not planning
- ☐ B Remove the default and enter plan mode deliberately for changes that warrant it
- ☐ C Keep the default but instruct Claude to produce shorter plans for small changes
- ☐ D Remove plan mode entirely and rely on reviewing the diff

◆ PAUSE AND DECIDE

What should they do?

- A

 Keep the default
"Always safer" ignores the cost. Anthropic's own guidance says skip the plan when the diff is one sentence.
- B

 Deliberate, not default
Keeps planning where it pays and removes it where it does not. The engineer decides, per change.
- C

 Shorter plans
Still a plan, still an approval step. It shrinks the overhead rather than removing it.
- D

 Remove it entirely
Overcorrects. Scenario 1 was the case where planning genuinely saved them.

THE SIGNAL

Paired with scenario 1 deliberately. Same engineer, same tool, opposite answer, because the change size flipped.

A team reports an intermittent bug: under some conditions a scheduled job processes the same record twice. They describe the symptom to Claude Code and ask it to fix it.

Claude makes a plausible change and reports the issue resolved. Two days later it recurs.

This has now happened three times with three different changes.

What is the most effective change to how they work?

- ☐ A Give Claude more detail about the conditions under which it occurs
- ☐ B Write a test that reproduces the double processing, and require it to fail before any fix
- ☐ C Use plan mode so the approach is reviewed before implementation
- ☐ D Ask Claude to explain why its fix works before accepting it

◆ PAUSE AND DECIDE

What is the most effective change to how they work?

- A

 More detail
Three attempts already failed on description. A fourth description is the same method with more words.
- B

 A reproducing test
Turns "reported resolved" into "provably resolved", and leaves a regression guard behind.
- C

 Plan mode
Reviews the approach. Nobody knew the approach was wrong, which is exactly the problem.
- D

 Explain the fix
A confident explanation of a wrong fix is what they have received three times already.

THE SIGNAL

"Reports the issue resolved" is the tell. Self-assessment with no external check, failing exactly as predicted.

An engineer asks Claude Code to "clean up the reporting module". Claude spends fifteen minutes making changes: renaming functions, extracting helpers, reordering methods and rewriting comments.

None of it is what the engineer wanted. They had meant "remove the three unused report types".

They want to avoid this kind of mismatch.

What is the most effective habit to adopt?

- ☐ A Have Claude ask clarifying questions before implementing when a request is open-ended
- ☐ B Add a CLAUDE.md section defining what "clean up" means for this team
- ☐ C Always use plan mode, so the misunderstanding surfaces in the plan
- ☐ D Review the diff more carefully before accepting changes

◆ PAUSE AND DECIDE

What is the most effective habit to adopt?

- A

 Interview first
The ambiguity was in the request. Two questions before starting would have caught it in seconds.
- B

 Define "clean up"
Fixes this phrase. The next vague request is a different phrase with the same failure.
- C

 Always plan mode
Genuinely helps here, and "always" is the over-planning trap from scenario 2.
- D

 Review more carefully
Catches it after fifteen minutes of wasted work rather than before.

THE SIGNAL

Option C is nearly right. Plan mode would surface it — but the generalisable habit is resolving ambiguity, not planning everything.

A team has a PreToolUse hook on Write and Edit that rejects any file containing a TODO comment, to keep the codebase clean.

Engineers report that during large multi-file changes Claude gets confused, sometimes reverting to earlier steps or losing track of what it has already done. It works fine on single-file edits.

The hook fires several times during a typical large change.

What is the root cause, and what should they do?

- ☐ A The context window is filling. Use context: fork for large changes
- ☐ B Blocking writes mid-task breaks multi-step reasoning. Move the check to PostToolUse or pre-commit
- ☐ C The hook is too slow. Make it asynchronous
- ☐ D Claude needs the TODO rule in CLAUDE.md so it stops writing them

◆ PAUSE AND DECIDE

What is the root cause, and what should they do?

A Context window

Plausible for large changes, and it does not explain why the hook firing correlates with the confusion.

C Async hook

An async hook cannot block, so the rule stops being enforced. It removes the symptom by removing the control.

B Mid-task blocking

Each rejected write interrupts the sequence and Claude loses its place. Validate after, not during.

D The rule in CLAUDE.md

Reduces how often the hook fires. It does not change what happens when it does.

THE SIGNAL

"Fine on single-file edits, confused on multi-file" is the tell. The problem is interruption of a sequence, not volume.